**UNIVERSIDAD DIEGO PORTALES**
INGENIERÍA CIVIL EN INFORMÁTICA Y TELECOMUNICACIONES

SISTEMAS OPERATIVOS • TAREA 2

# INFORME TÉCNICO
# Torneo de Komikan Concurrente

Diseño de sincronización, riesgos de concurrencia y evaluación

**Integrantes:** Thiare Saavedra y Yomar Perez **Fecha:** 04/11/2026
**Docente:** Yerko Ortiz

## 1. Diseño del esquema de sincronización

**¿Cómo se organiza el sistema?** La solución está programada en C17 con hilos POSIX (*pthreads*). El **servidor** representa a cada jugador con un hilo, administra una cola FIFO (primero en llegar, primero en salir), forma parejas Puma–Perros y mantiene como máximo $M$ mesas activas. Si no hay mesa disponible, el emparejador espera con `pthread_cond_wait()`, sin consumir CPU en consultas repetidas. Cada partida tiene **13 hilos de fichas** (un Puma y doce Perros) y **un hilo juez** independiente.

**¿Qué recursos se comparten y cómo se protegen?** Un **mutex** funciona como un candado: mientras un hilo lo mantiene, otro no puede entrar a esa sección del código. El mutex `servidor->mutex` protege la cola, los estados de jugadores y la asignación de mesas. En cada partida, `mesa->mutex` protege el tablero, las propuestas pendientes y el estado de finalización. Las variables de condición permiten que un hilo espere hasta que cambie algo importante; al esperar, libera temporalmente el mutex para que otro hilo pueda avanzar. `mesa_crear()` inicializa el tablero y las primitivas antes de poner a trabajar a los hilos de la mesa.

**¿Cómo garantizamos la exclusión mutua?** Las fichas *no escriben directamente* en el tablero: cada una envía una propuesta de movimiento o captura. El juez procesa las propuestas una por una y, mientras mantiene el mutex de la mesa, verifica si el origen sigue siendo válido, si existe conexión con el destino, si está libre y, cuando corresponde, si la captura es legal. Solo entonces actualiza el tablero. **Ejemplo:** si dos Perros intentan ocupar la misma casilla, el juez acepta la primera propuesta válida; al evaluar la segunda, encuentra el destino ocupado y la rechaza. Así se evita que dos fichas ocupen la misma posición. Cada actualización del tablero se realiza como una operación indivisible frente a las otras propuestas.

**¿Quién informa los resultados y libera los recursos?** El juez detecta las victorias (seis capturas del Puma o inmovilización del Puma), publica el resultado y registra movimientos y tableros actualizados. Para no mantener bloqueada la partida mientras escribe, copia el tablero y hace el registro después de liberar `mesa->mutex`. Un mutex propio del módulo de log evita mezclar los mensajes de distintas mesas. Ante una terminación *recuperable* de un hilo de jugador, su rutina de limpieza avisa a la mesa; **el juez procesa ese aviso** y declara ganador al rival por abandono. Ante Ctrl+C, un hilo con `sigwait()` solicita detener llegadas y cancelar partidas. Después se recogen los hilos con `pthread_join()` y se liberan las estructuras y primitivas cuando ya nadie las utiliza.

## 2. Análisis de riesgos de concurrencia

**Condición de carrera (*race condition*).** Se produciría si dos fichas cambiaran el tablero a la vez, por ejemplo al intentar tomar una misma casilla libre. **Solución:** las fichas solo proponen; el juez valida y modifica el tablero bajo un único mutex. También descarta propuestas que quedaron desactualizadas.

**Interbloqueo (*deadlock*).** Ocurre cuando varios hilos quedan esperando recursos entre sí y ninguno puede continuar. Por ejemplo, sería peligroso bloquear primero una casilla y luego otra mientras otro hilo lo hace al revés. **Prevención:** no se usan mutexes por casilla ni se necesitan varios bloqueos para una captura: cada mesa tiene un mutex principal. La espera con condiciones suelta el mutex mientras el hilo duerme y el programa espera a que terminen los hilos antes de destruirlo. Así se reducen las posibilidades de espera circular, aunque una falla fuera de los puntos de limpieza previstos sigue siendo un riesgo.

**Bloqueo activo (*livelock*).** Los hilos podrían seguir intentando movimientos, pero rechazarse repetidamente sin avanzar en el juego. **Mitigación:** cada ficha espera la decisión de su propuesta antes de enviar otra y usa retardos aleatorios; no existe un ciclo de reintentos inmediatos que acapare CPU. Esto reduce el problema, pero

no garantiza por sí solo que toda partida termine rápidamente.

**Inanición (*starvation*).** Un jugador o una ficha puede recibir menos oportunidades que los demás. **Mitigación:** la cola FIFO atiende a los jugadores por llegada y el juez procesa las propuestas pendientes por orden de recepción; cuando una mesa se libera, se despierta al emparejador. **Límite:** no hay garantía formal de un tiempo máximo de espera, pues intervienen el sistema operativo y la duración de las partidas.

## 3. Evaluación y justificación de las decisiones

**¿Por qué un juez y un mutex por mesa?** Esta opción es más sencilla de comprender y probar que bloquear cada posición por separado. Una captura afecta varias casillas, por lo que varios mutexes aumentarían la complejidad y los riesgos de interbloqueo. El costo es que, **dentro de una misma mesa, las actualizaciones del tablero se procesan de una en una**; aun así, las fichas deciden y esperan de manera independiente y pueden funcionar hasta $M$ partidas simultáneas.

**¿Qué ocurre con el rendimiento?** El *throughput* es la cantidad de partidas terminadas por segundo; la *latencia* es el tiempo que tarda una propuesta en recibir respuesta. En una prueba exploratoria con $N = 40$, retardos configurados de 1 a 3 ms y tres repeticiones por escenario, se obtuvieron:

| Límite de mesas | Tiempo medio | Partidas/s (aprox.) | Mesas en paralelo |
|---|---|---|---|
| $M = 1$ | 0,69 s | aprox. 29 | 1 |
| $M = 4$ | 0,28 s | aprox. 72 | 4 |

Al aumentar las mesas, más partidas avanzaron en paralelo y aumentó el throughput observado, a cambio de utilizar más hilos y memoria. La **latencia por movimiento no se midió de forma directa**: depende del retardo configurado, la planificación de los hilos y la espera al juez. Registrar el tablero fuera del mutex ayuda a evitar esperas adicionales.

**¿Qué límites tiene nuestra solución?** Aunque el sistema permite ejecutar varias partidas simultáneamente, cada mesa cuenta con un único juez que procesa los movimientos de forma secuencial. Esto garantiza la consistencia del tablero, pero limita la cantidad de movimientos que pueden procesarse al mismo tiempo dentro de una partida. Además, como se crea un hilo por jugador, la cantidad de participantes depende de la memoria y de los recursos disponibles en el sistema operativo. Por otra parte, utilizar una semilla fija no garantiza resultados idénticos, ya que el orden de ejecución de los hilos puede variar. Tampoco existe un tiempo máximo garantizado para finalizar una partida, y un fallo fatal como `SIGSEGV` podría terminar todo el proceso. Finalmente, aunque las pruebas realizadas no detectaron errores de memoria ni problemas de sincronización en los escenarios evaluados, esto no garantiza que el programa esté libre de fallos en cualquier situación.